

REVISÃO DE SQL

Usando como base ORACLE HR

1. Estrutura básica de uma consulta SQL

A estrutura mais comum de uma consulta é:

```
SELECT coluna1,  
        coluna2,  
        coluna3  
FROM tabela  
WHERE condição  
GROUP BY coluna  
HAVING condição_do_grupo  
ORDER BY coluna;
```

Ordem lógica de execução

Embora escrevamos o comando nesta ordem:

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY
```

o banco de dados trabalha conceitualmente na seguinte sequência:

```
FROM / JOIN  
↓  
WHERE  
↓  
GROUP BY  
↓  
HAVING  
↓  
SELECT  
↓  
ORDER BY
```

Essa sequência ajuda a entender principalmente a diferença entre `WHERE` e `HAVING`.

2. SELECT — escolher os dados

O `SELECT` define quais informações serão apresentadas.

```
SELECT first_name,  
       last_name,  
       salary  
FROM hr.employees;
```

Podemos seleccionar todas as columnas:

```
SELECT * FROM hr.employees;
```

Porém, em exercícios e sistemas reais, é preferível informar explicitamente as colunas necessárias.

3. FROM — indicar a tabela

O FROM informa de onde os dados serão obtidos.

```
SELECT first_name,  
       last_name  
FROM hr.employees;
```

Neste exemplo:

```
FROM hr.employees
      |
      |
      |----- tabela
      |
      |----- schema/usuário
```

4. Alias — apelido para tabelas

Quando trabalhamos com várias tabelas, é recomendado utilizar aliases.

Exemplo: `SELECT e.first name, e.last name FROM hr.employees e;`

Neste caso: `e = hr.employees`

Podemos utilizar alias diferentes:

```
FROM hr.employees e
JOIN hr.departments d
```

Então: e = employees d = departments

Isso torna a consulta mais curta e facilita a leitura.

5. JOIN — relacionando tabelas

O `JOIN` permite combinar informações que estão armazenadas em tabelas diferentes.

Exemplo:

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name  
FROM hr.employees e  
JOIN hr.departments d  
     ON e.department_id = d.department_id;
```

A lógica é:

```
EMPLOYEES  
  |  
  | department_id  
  |  
  ↓  
DEPARTMENTS
```

O ponto mais importante é identificar **qual coluna relaciona uma tabela à outra**.

6. ON — condição do relacionamento

O `ON` define como as tabelas serão relacionadas.

```
JOIN hr.departments d  
ON e.department_id = d.department_id
```

Pergunta que devemos fazer:

Qual coluna da primeira tabela corresponde à coluna da segunda tabela?

Exemplo:

```
employees.department_id  
      =  
departments.department_id
```

7. Navegação entre várias tabelas

Podemos utilizar vários `JOINS` na mesma consulta.

Exemplo:

```
FROM hr.employees e

JOIN hr.departments d
ON e.department_id = d.department_id

JOIN hr.locations l
ON d.location_id = l.location_id

JOIN hr.countries c
ON l.country_id = c.country_id
```

Observe a navegação:

```
EMPLOYEES
  ↓
DEPARTMENTS
  ↓
LOCATIONS
  ↓
COUNTRIES
```

Cada `JOIN` permite avançar para outra tabela relacionada.

8. INNER JOIN

Quando escrevemos:

```
JOIN
```

estamos utilizando, por padrão, um:

```
INNER JOIN
```

Exemplo:

```
SELECT e.first_name,
       d.department_name
FROM hr.employees e
INNER JOIN hr.departments d
ON e.department_id = d.department_id;
```

Serão retornados somente registros que possuem correspondência nas duas tabelas.

9. LEFT JOIN

O `LEFT JOIN` mantém todos os registros da tabela da esquerda, mesmo quando não existe correspondência na tabela da direita.

Exemplo:

```
SELECT e.first_name,  
       m.first_name AS gerente  
FROM hr.employees e  
LEFT JOIN hr.employees m  
ON e.manager_id = m.employee_id;
```

Esse é um exemplo de **SELF JOIN**.

10. SELF JOIN

Um `SELF JOIN` ocorre quando uma tabela é relacionada com ela mesma.

No exercício dos gerentes:

```
FROM hr.employees e  
LEFT JOIN hr.employees m  
ON e.manager_id = m.employee_id
```

Temos:

```
e = funcionário  
m = gerente
```

A mesma tabela `EMPLOYEES` assume dois papéis diferentes.

```
EMPLOYEE  
  |  
  | manager_id  
  ↓  
EMPLOYEE
```

11. WHERE — filtrar registros

O `WHERE` filtra registros antes do agrupamento.

Exemplo:

```
SELECT first_name,  
       salary  
FROM hr.employees  
WHERE salary > 5000;
```

Somente funcionários com salário superior a 5000 serão considerados.

Podemos utilizar:

```
=  
<>  
>  
<  
>=  
<=
```

Exemplos:

```
WHERE salary >= 5000  
  
WHERE department_id = 10  
  
WHERE salary <> 3000
```

12. Operadores AND e OR

Podemos combinar condições.

AND

As duas condições precisam ser verdadeiras:

```
WHERE salary > 5000  
AND department_id = 10;
```

OR

Uma das condições precisa ser verdadeira:

```
WHERE department_id = 10  
OR department_id = 20;
```

Atenção aos parênteses

Quando usamos AND e OR juntos, é recomendado utilizar parênteses:

```
WHERE (department_id = 10 OR department_id = 20)  
AND salary > 5000;
```

Isso evita interpretações incorretas da condição.

13. IS NULL

Para verificar se um campo não possui valor:

```
WHERE manager_id IS NULL;
```

Para verificar se possui valor:

```
WHERE manager_id IS NOT NULL;
```

Não devemos utilizar:

```
WHERE manager_id = NULL
```

O correto é:

```
WHERE manager_id IS NULL
```

14. GROUP BY — agrupamento

O `GROUP BY` reúne registros que possuem o mesmo valor em determinada coluna.

Exemplo:

```
SELECT department_id,  
       COUNT(employee_id)  
FROM hr.employees  
GROUP BY department_id;
```

Resultado conceitual:

```
Departamento 10 → 6 funcionários  
Departamento 20 → 7 funcionários  
Departamento 30 → 6 funcionários
```

O `GROUP BY` é muito utilizado junto com funções de agregação.

15. COUNT — contar registros

Para contar funcionários:

```
COUNT(employee_id)
```

Exemplo:

```
SELECT department_id,  
       COUNT(employee_id) AS quantidade_funcionarios  
FROM hr.employees  
GROUP BY department_id;
```

O `AS` cria um nome mais amigável para a coluna do resultado.

16. SUM — somar valores

Para somar salários:

```
SUM(salary)
```

Exemplo:

```
SELECT department_id,  
       SUM(salary) AS total_salarios  
FROM hr.employees  
GROUP BY department_id;
```

17. AVG — calcular média

Para calcular o salário médio:

```
AVG(salary)
```

Exemplo:

```
SELECT department_id,  
       AVG(salary) AS salario_medio  
FROM hr.employees  
GROUP BY department_id;
```

18. MIN e MAX

Menor valor

```
MIN(salary)
```

Maior valor

```
MAX(salary)
```

Podemos utilizar os dois:

```
SELECT department_id,  
       MIN(salary) AS menor_salario,  
       MAX(salary) AS maior_salario  
FROM hr.employees  
GROUP BY department_id;
```

19. Várias funções de agregação

Podemos utilizar várias funções na mesma consulta:

```
SELECT department_id,  
       COUNT(employee_id) AS quantidade,  
       MIN(salary) AS menor_salario,  
       MAX(salary) AS maior_salario,  
       AVG(salary) AS salario_medio,  
       SUM(salary) AS total_salarios  
FROM hr.employees  
GROUP BY department_id;
```

Esse conceito aparece diretamente no exercício 15.

20. HAVING — filtrar grupos

O `HAVING` é utilizado para filtrar resultados depois do `GROUP BY`.

Exemplo:

```
SELECT department_id,  
       COUNT(employee_id) AS quantidade  
FROM hr.employees  
GROUP BY department_id  
HAVING COUNT(employee_id) > 5;
```

Aqui estamos dizendo:

Agrupe os funcionários por departamento e mostre somente os departamentos com mais de 5 funcionários.

21. WHERE x HAVING

Esta é uma das diferenças mais importantes da lista.

WHERE

Filtra registros:

```
WHERE salary > 5000
```

HAVING

Filtra grupos:

```
HAVING AVG(salary) > 8000
```

Exemplo completo:

```
SELECT department_id,  
       AVG(salary) AS salario_medio  
FROM hr.employees  
WHERE salary > 3000  
GROUP BY department_id  
HAVING AVG(salary) > 8000;
```

A lógica é:

WHERE

↓

filtra funcionários

GROUP BY

↓

agrupa por departamento

HAVING

↓

filtra os departamentos

22. ORDER BY — ordenar resultados

Para ordenar:

```
ORDER BY salary;
```

Por padrão, a ordem é crescente.

Também podemos escrever:

```
ORDER BY salary ASC;
```

Para ordem decrescente:

```
ORDER BY salary DESC;
```

23. Ordenar utilizando alias

Podemos ordenar pelo nome criado no SELECT.

Exemplo:

```
SELECT department_id,  
       SUM(salary) AS total_salarios  
FROM hr.employees  
GROUP BY department_id  
ORDER BY total_salarios DESC;
```

24. Alias de colunas — AS

Podemos dar nomes mais claros às colunas.

```
COUNT(employee_id) AS quantidade_funcionarios  
SUM(salary) AS total_salarios  
AVG(salary) AS salario_medio
```

O AS melhora a apresentação do resultado.

25. Distinct — eliminar repetições

Para listar valores sem repetição:

```
SELECT DISTINCT department_id  
FROM hr.employees;
```

Por exemplo:

```
SELECT DISTINCT job_id  
FROM hr.employees;
```

Retorna cada cargo uma única vez.

26. Construção de uma consulta passo a passo

Uma boa estratégia para resolver os exercícios é não tentar escrever tudo de uma vez.

Passo 1 — identificar as tabelas

Pergunte:

Quais tabelas possuem as informações solicitadas?

Passo 2 — identificar os relacionamentos

Pergunte:

Qual coluna liga essas tabelas?

Passo 3 — montar o FROM

```
FROM hr.employees e
```

Passo 4 — adicionar os JOINS

```
JOIN hr.departments d  
ON e.department_id = d.department_id
```

Passo 5 — escolher as colunas

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name
```

Passo 6 — aplicar filtros

```
WHERE ...
```

Passo 7 — agrupar, quando necessário

```
GROUP BY ...
```

Passo 8 — filtrar grupos

```
HAVING ...
```

Passo 9 — ordenar

```
ORDER BY ...
```

27. Modelo para exercícios com JOIN

Use como referência:

```
SELECT
    tabela1.coluna,
    tabela2.coluna
FROM tabela1
JOIN tabela2
ON tabela1.chave = tabela2.chave;
```

28. Modelo para exercícios com vários JOINS

```
SELECT
    e.first_name,
    d.department_name,
    l.city,
    c.country_name

FROM hr.employees e

JOIN hr.departments d
ON e.department_id = d.department_id

JOIN hr.locations l
ON d.location_id = l.location_id

JOIN hr.countries c
ON l.country_id = c.country_id;
```

29. Modelo para COUNT + GROUP BY

```
SELECT
    coluna_agrupamento,
    COUNT(coluna) AS quantidade

FROM tabela

GROUP BY coluna_agrupamento;
```

30. Modelo para SUM + GROUP BY

```
SELECT
    coluna_agrupamento,
    SUM(valor) AS total

FROM tabela

GROUP BY coluna_agrupamento;
```

31. Modelo para análise estatística

```
SELECT
    coluna_agrupamento,
    COUNT(id) AS quantidade,
    MIN(valor) AS menor_valor,
    MAX(valor) AS maior_valor,
    AVG(valor) AS media,
    SUM(valor) AS total

FROM tabela

GROUP BY coluna_agrupamento;
```

32. Modelo para GROUP BY + HAVING

```
SELECT
    coluna_agrupamento,
    COUNT(id) AS quantidade

FROM tabela

GROUP BY coluna_agrupamento

HAVING COUNT(id) > 5;
```

33. Modelo completo

Uma consulta mais completa pode seguir este padrão:

```
SELECT
    coluna1,
    coluna2,
    COUNT(id) AS quantidade,
    SUM(valor) AS total,
    AVG(valor) AS media

FROM tabela1 t1

JOIN tabela2 t2
ON t1.codigo = t2.codigo

WHERE condição

GROUP BY
    coluna1,
    coluna2

HAVING COUNT(id) > 5

ORDER BY total DESC;
```

34. Mapa dos 20 exercícios

Exercícios	Principais conceitos
1	SELECT + JOIN
2	SELECT + JOIN
3	JOIN entre 3 tabelas
4	SELF JOIN + LEFT JOIN
5	JOIN entre 4 tabelas
6	COUNT + GROUP BY
7	COUNT + GROUP BY
8	JOIN + COUNT + GROUP BY
9	4 tabelas + COUNT + GROUP BY
10	COUNT + SUM + GROUP BY
11	COUNT + SUM + GROUP BY
12	JOIN + COUNT + SUM
13	GROUP BY + HAVING
14	GROUP BY + SUM + HAVING
15	COUNT + MIN + MAX + AVG + SUM
16	SELF JOIN + COUNT + GROUP BY
17	AVG + GROUP BY
18	MIN + MAX + AVG
19	AVG + GROUP BY + HAVING
20	JOIN + COUNT + GROUP BY + ORDER BY

35. Checklist para resolver os exercícios

Antes de executar uma consulta, verifique:

1. Quais dados preciso mostrar?

Identifique as colunas do `SELECT`.

2. Em quais tabelas estão os dados?

Identifique as tabelas do `FROM` e dos `JOINS`.

3. Como as tabelas estão relacionadas?

Identifique as chaves utilizadas no `ON`.

4. Preciso filtrar registros?

Utilize:

`WHERE`

5. Preciso contar, somar ou calcular média?

Utilize:

`COUNT ()`
`SUM ()`
`AVG ()`
`MIN ()`
`MAX ()`

6. Preciso agrupar os resultados?

Utilize:

`GROUP BY`

7. Preciso filtrar o resultado do agrupamento?

Utilize:

`HAVING`

8. Preciso ordenar?

Utilize:

`ORDER BY`

9. Existem registros que podem não possuir correspondência?

Avalie se deve utilizar:

LEFT JOIN

em vez de:

INNER JOIN

36. Principais comandos para memorizar

SELECT	→ escolhe as colunas
FROM	→ informa a tabela
JOIN	→ relaciona tabelas
ON	→ define o relacionamento
WHERE	→ filtra registros
GROUP BY	→ agrupa registros
HAVING	→ filtra grupos
ORDER BY	→ ordena resultados
AS	→ cria alias
DISTINCT	→ elimina repetições

Funções de agregação

COUNT()	→ quantidade
SUM()	→ soma
AVG()	→ média
MIN()	→ menor valor
MAX()	→ maior valor

37. Regra prática para lembrar

Uma forma simples de raciocinar durante os exercícios:

QUAIS DADOS?
↓
SELECT

DE ONDE?
↓
FROM

COMO RELACIONAR?
↓
JOIN / ON

QUAIS REGISTROS?
↓
WHERE

COMO AGRUPAR?
↓
GROUP BY

QUAIS GRUPOS?
↓
HAVING

COMO ORDENAR?
↓
ORDER BY